
Génie Logiciel

CER | Burnout

Lundi 26 janvier 2026

TABLE DES MATIÈRES

Étude de cas : Burnout	3
AAV	3
Mots clés	4
Contexte	4
Problématique	4
Contraintes	4
Généralisation	5
Livrables	5
Pistes de solutions	5
Plan d'actions	5
Démarche : Burnout	6
Définition: Mots clés	6
L'écosystème .NET : Framework vs Core	9
L'architecture interne : CLR et CoreCLR	10
Conteneurisation et Docker	11
Intégration Continue (CI) et Git	11
Corbeille d'Exercice - Git	12
Initialisation d'un dépôt Git	12
Gestion des branches	12
Résolution de conflits	13
Utilisation d'un dépôt distant	13
Revert et Reset	14
Visualiser l'historique des commits	14
Stashing des modifications	15
Création et utilisation de tags	15
Configuration de Git	16
Ignorer des fichiers avec .gitignore	16
Collaboration avec des Pull Requests	16
Rebase vs Merge	17
Nettoyage du dépôt avec git gc	17
Utilisation avancée de git log	18
Configuration avancée de Git	18
Workshop	19
A1. Notion de classe	19
A2. Notion d'objets / Instances	19
A3. Notion de visibilité	19
A4. Notion d'encapsulation (Propriétés)	20
A5. Les Records	20
A6. Types Valeur (Struct) vs Référence (Class)	20
B1. Constructeurs	21

B2. Paramètres optionnels	21
B3. Références et Garbage Collector	21
B4. La directive using	21
C1. Membres statiques	22
C2. LINQ (Language Integrated Query)	22
D1. Héritage	23
D2. Polymorphisme (Virtual / Override)	23
D3. Classes Abstraites	23
D4. Les Interfaces	24
E1. Async / Await	24
Analyse et résolution : Burnout	25
Conclusion	25
Capitalisation	26
Ressources	27

ÉTUDE DE CAS : BURNOUT

AAV

- 1 Décrire .NET Core et .NET Framework et leur rôle dans .Net
- 1 Décrire les architectures .NET (Core et Framework)
- 2 Expliquer les évolutions entre .NET Core et .NET Framework
- 2 Expliquer les notions de CoreCLR et CLR (architecture, composants, processus d'exécution, gestion automatique de la mémoire)
- 2 Expliquer les notions d'assemblage pour .NET Core et .NET Framework (déploiement, GAC)
- 1 Connaître les principaux outils supportant la chaine d'intégration continue
- 1 Connaître les principes généraux de la conteneurisation et ses avantages en terme de déploiement de solutions
- 1 Connaître les environnements et outils Docker
- 3 Pratiquer l'installation et la configuration de Git
- 3 Relier Visual Studio à un dépôt Git
- 3 Illustrer les principes généraux de gestion de versions pour un fichier et Appliquer les commandes Git associées

MOTS CLÉS

- POO
- Génie logiciel
- Modularité
- Build
- Versionning
- Tests unitaires
- Classes
- Objets
- Déploiement
- Application
- Régression
- Fichier source

CONTEXTE

L'entreprise 2F Roaming s'est étendu de 2 développeurs à une équipe de 20 personnes, tout en conservant des méthodes de travail obsolètes. Ce manque de structuration a engendré des difficultés majeures, des problèmes de qualité, de productivité et de gestion des versions. Face à ces défis, Mathilde a été mandatée pour moderniser les pratiques de développement et instaurer une culture DevOps au sein de l'équipe.

PROBLÉMATIQUE

Comment améliorer les pratiques de développement logiciel chez 2F Roaming pour garantir la qualité, la productivité et la gestion efficace des versions dans un environnement en croissance rapide ?

CONTRAINTES

- POO
- Automatisation des tests
- IDE compatible avec POO
- Multi-plateforme

GÉNÉRALISATION

- POO
- Génie logiciel

LIVRABLES

- Plan d'amélioration
 - Environnement de développement
 - Gestion de versions
 - Colaboration
 - Tests automatisés

PISTES DE SOLUTIONS

- Se renseigner sur les solutions de gestion de versions (Git, SVN)
- Etudier les test automatisés
- Mettre en place un environnement de développement standardisé
- Former l'équipe aux nouvelles pratiques

PLAN D' ACTIONS

- Analyse des besoins et des pratiques actuelles
- Recherche et sélection des outils adaptés
 - Gestion de versions (Git)
 - Intégration continue (Jenkins, GitLab CI)
 - Tests automatisés (JUnit, Selenium)
 - Environnement de développement (IDE, conteneurs)
- Mise en place de l'environnement de développement
- Documentation des nouvelles pratiques

DÉMARCHE : BURNOUT

DÉFINITION : MOTS CLÉS

POO: La Programmation Orientée Objet (**POO**) est un paradigme de programmation qui utilise des "objets" pour représenter des données et des méthodes. Un **objet** est une **instance** d'une **classe**, qui peut être considérée comme un modèle ou un plan définissant les attributs et les comportements de l'objet.

Les classes : Une classe est une structure qui définit les attributs et les méthodes que ses objets peuvent posséder. Elle sert de plan pour créer des objets.

```
public class Voiture
{
    private string marque;
    private string modele;

    public string Marque
    {
        get { return marque; }
        set { marque = value; }
    }
    public string Modele
    {
        get { return modele; }
        set { modele = value; }
    }

    public Voiture(string marque, string modele)
    {
        this.marque = marque;
        this.modele = modele;
    }

    public void Demarrer()
    {
        Console.WriteLine($"{Marque} {Modele} démarre.");
    }
}
```

/ Exemple de classe "Voiture" avec des attributs privés et des propriétés publiques */*

Les objets : Un objet est une instance d'une classe. Il contient des données et des méthodes définies par la classe.

Les attributs : En Python, les attributs d'un objet englobent à la fois les variables (ou propriétés) et les méthodes. Tout ce qui est associé à une instance ou à la classe elle-même est considéré comme un attribut.

Les méthodes : Les méthodes sont des fonctions définies à l'intérieur d'une classe. Elles décrivent les comportements des objets de cette classe.

L'encapsulation : L'encapsulation est le principe de restreindre l'accès direct à certaines données d'un objet et de contrôler cet accès via des méthodes. Cela protège les données internes de l'objet contre les modifications non autorisées. Pour cela, on utilise un underscore _ devant le nom de l'attribut.

L'héritage : L'héritage permet à une classe de dériver d'une autre classe, en héritant de ses attributs et méthodes. Cela favorise la réutilisation du code et la création de hiérarchies de classes.

Polymorphisme : Le polymorphisme permet à des objets de classes différentes d'être traités comme des objets de la même classe via des interfaces communes. Cela permet d'utiliser des méthodes dans une interface commune sans connaître les détails des classes spécifiques.

Abstraction : L'abstraction consiste à créer des classes abstraites qui définissent des méthodes communes sans implémenter leur logique. Ce sont les classes dérivées qui implémentent ces méthodes.

A quoi sert la POO ?

- **Réutilisation du code** : Grâce à l'héritage, le code peut être réutilisé, ce qui réduit la duplication et facilite la maintenance.
- **Modularité** : La POO permet de diviser un programme en morceaux indépendants (classes et objets), rendant le code plus facile à comprendre et à maintenir.
- **Extensibilité** : Les programmes orientés objet sont plus faciles à étendre. De nouvelles fonctionnalités peuvent être ajoutées en créant de nouvelles classes ou en modifiant les classes existantes sans affecter le reste du programme.
- **Encapsulation** : L'encapsulation protège les données et assure une manipulation cohérente et sécurisée des attributs de l'objet.

Génie logiciel: Est appelé **génie logiciel** l'application des principes de l'**ingénierie** (traitant habituellement des **systèmes physiques**) à la **conception**, au **développement**, au **test**, au **déploiement** et à la **gestion de logiciels**.

Cette discipline applique au **développement logiciel** l'approche **structurée** et **hiérarchisée** de la programmation utilisée par l'ingénierie, dans le but d'améliorer la **qualité**, les **délais** et le **budget**, tout en assurant des **tests ordonnés** et la **certification des ingénieurs**.

API: Une API (application programming interface ou « interface de programmation d'application ») est une interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités. Les API offrent de nombreuses possibilités, comme la portabilité des données, la mise en place de campagnes de courriels publicitaires, des programmes d'affiliation, l'intégration de fonctionnalités d'un site sur un autre ou l'open data. Elles peuvent être gratuites ou payantes.

Modularité: C'est l'étape de transformation. On prend les fichiers sources (la recette) et on les compile/prépare pour qu'ils deviennent un fichier exécutable par l'ordinateur. C'est le moment où la recette devient un plat prêt à être servi.

Build: C'est l'étape de transformation. On prend les fichiers sources (la recette) et on les compile/prépare pour qu'ils deviennent un fichier exécutable par l'ordinateur. C'est le moment où la recette devient un plat prêt à être servi.

Versionning: C'est la machine à remonter le temps du code. Via des outils comme Git, on enregistre chaque modification. Si on fait une erreur catastrophique aujourd'hui, on peut revenir exactement à l'état du code d'hier à 14h02. C'est aussi ce qui permet à 100 développeurs de travailler sur le même fichier sans s'écraser les uns les autres.

Tests unitaires: Ce sont des petits gardiens automatisés. Un test unitaire vérifie une seule petite fonctionnalité (une "unité") de manière isolée.

Exemple : On écrit un test pour vérifier que la fonction "Addition" renvoie bien 4 quand on lui donne 2+2. Si demain elle renvoie 5, le test passe au rouge.

Déploiement: C'est l'étape où l'on envoie le code (après le build et les tests) sur les serveurs réels pour que les utilisateurs puissent y accéder. Passer de "ça marche sur mon ordi" à "c'est disponible sur Internet".

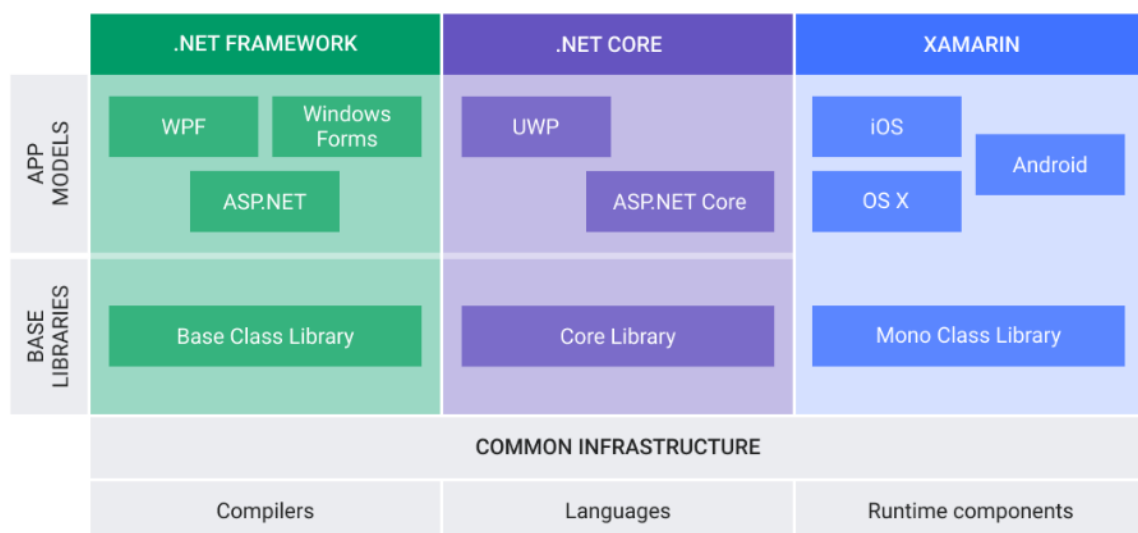
Application: C'est le résultat final, l'ensemble complet et fini. C'est l'outil avec lequel l'utilisateur interagit (ton application Instagram, ton logiciel de compta, etc.). C'est la somme de tous les modules, objets et builds mentionnés plus haut.

Régression: Il y a "régression" quand on répare un bug à gauche, mais que cela en crée un nouveau à droite (ou qu'un ancien bug réapparaît). C'est pour éviter cela qu'on fait des tests : pour s'assurer qu'ajouter une fonctionnalité ne fait pas reculer la qualité globale.

Fichier source: C'est le document de base écrit par le développeur. Imagine une recette de cuisine écrite sur papier : c'est du texte lisible par l'humain (en Python, Java, C++, etc.), mais ce n'est pas encore le plat final. C'est la matière première.

L'ÉCOSYSTÈME .NET : FRAMEWORK VS CORE

Aujourd'hui, on parle simplement de .NET (5, 6, 7, 8) qui est la fusion et l'évolution de tout ce qui suit.



- **.NET Framework (L'ancêtre)** : Conçu exclusivement pour Windows. C'est le moteur historique pour les applications de bureau (WPF, WinForms) et le Web ancien (ASP.NET WebForms).
- **.NET Core (Le renouveau)** : Une version réécrite de zéro, cross-platform (Windows, Linux, macOS). Il est beaucoup plus rapide, léger et conçu pour le Cloud et les micro-services.
- **Leur rôle** : Fournir une bibliothèque de code (Base Class Library) et un environnement d'exécution pour que ton code tourne de la même manière partout.

Le passage du Framework au Core a marqué trois ruptures :

- **Performance** : .NET Core est drastiquement plus rapide.
- **Modularité** : Contrairement au Framework qui est un bloc monolithique installé sur Windows, .NET Core est distribué via des paquets NuGet. On n'embarque que ce dont on a besoin.
- **Open Source** : .NET Core est totalement ouvert à la communauté, contrairement au Framework original.

L'ARCHITECTURE INTERNE : CLR ET CORECLR

Le CLR (Common Language Runtime) est le cœur du moteur. C'est lui qui "exécute" le code.

- **Composants & Processus d'exécution**
 - 1. Ton code C# est compilé en CIL (Common Intermediate Language).
 - 2. Le CLR prend ce CIL et le transforme en code machine (0 et 1).
 - 3. La compilation JIT (Just-In-Time).
- **Gestion de la mémoire (Garbage Collector)** : Tu n'as pas à détruire les objets manuellement. Le Garbage Collector (GC) surveille les objets qui ne sont plus utilisés et libère la mémoire automatiquement pour éviter les fuites.

Les Assemblages (Assembly) Un assemblage est le résultat du build (souvent un .dll ou un .exe).

- **Sur .NET Framework** : On utilisait souvent le GAC (Global Assembly Cache), un dossier centralisé dans Windows où toutes les applications piochaient leurs bibliothèques. C'était parfois le "DLL Hell" (conflits de versions).
- **Sur .NET Core** : Le GAC n'existe plus. Chaque application emporte ses propres dépendances avec elle (déploiement autonome), garantissant qu'elle fonctionnera partout sans polluer le système.

CONTENEURISATION ET DOCKER

La conteneurisation (avec Docker) consiste à emballer ton application avec tout ce dont elle a besoin (OS minimal, runtime .NET, librairies) dans une "boîte" isolée appelée Conteneur.

- **Avantage** : "Ça marche sur mon ordi, donc ça marchera sur le serveur". On élimine les problèmes d'environnement.
- **Outils** : **Docker Desktop** (pour développer), Docker Hub (pour stocker les images) et les fichiers Dockerfile (la recette de fabrication de l'image).

INTÉGRATION CONTINUE (CI) ET GIT

La chaîne d'intégration (CI) C'est l'automatisation. Dès que tu pousses du code sur Git, des outils s'activent pour vérifier que tout est OK :

- **Azure DevOps / GitHub Actions / GitLab CI** : Les chefs d'orchestre.
- **SonarQube** : Analyse la qualité du code.
- **Tests automatiques** : Exécution des tests unitaires définis plus haut.

Maîtriser Git (Commandes clés) Pour utiliser Git avec Visual Studio, on relie généralement l'IDE à un dépôt distant (GitHub/Azure Repos) via l'onglet "Git Changes".

Action	Commande Git	Explication
Initialiser	<code>git init</code>	Prépare le dossier pour le versionning.
Staging	<code>git add .</code>	Met les changements dans le "panier" avant de valider.
Valider	<code>git commit -m "msg"</code>	Enregistre une version (un point de sauvegarde).
Envoyer	<code>git push</code>	Envoie tes sauvegardes sur le serveur distant.
Récupérer	<code>git pull</code>	Récupère le travail des collègues.

Table 1: Commandes de base Git

CORBEILLE D'EXERCICE - GIT

INITIALISATION D'UN DÉPÔT GIT

Objectif : Initialiser un dépôt Git local et effectuer les premières opérations de base.4

1. Ouvrez votre terminal ou invite de commandes.
2. Naviguez jusqu'au répertoire où vous souhaitez créer un nouveau projet.
3. Exécutez la commande suivante pour initialiser un dépôt Git :

```
git init MonProjet
```

4. Accédez au répertoire du projet :

```
cd MonProjet
```

5. Créez un fichier README.md avec une description de votre projet.
6. Ajoutez le fichier au suivi Git :

```
git add README.md
```

7. Effectuez votre premier commit :

```
git commit -m "Premier commit : ajout du README"
```

GESTION DES BRANCHES

Objectif : Créer et gérer des branches dans un dépôt Git.

1. Depuis le répertoire de votre projet, créez une nouvelle branche nommée "develop" :

```
git branch develop
```

2. Basculez sur la branche "develop" :

```
git checkout develop
```

3. Modifiez le fichier README.md en ajoutant une nouvelle section.

4. Ajoutez et validez vos modifications :

```
git add README.md
```

```
git commit -m "Ajout d'une section dans le README sur la branche develop"
```

5. Revenez à la branche principale (main) :

```
git checkout main
```

6. Fusionnez les modifications de la branche "develop" dans "main" :

```
git merge develop
```

7. Vérifiez l'état de votre dépôt après la fusion :

```
git status
```

RÉSOLUTION DE CONFLITS

Objectif : Simuler et résoudre un conflit de fusion dans Git.

1. Sur la branche "main", modifiez le fichier README.md en ajoutant une ligne différente de celle que vous avez ajoutée sur la branche "develop".

2. Ajoutez et validez vos modifications sur la branche "main" :

```
git add README.md
git commit -m "Modification du README sur la branche main"
```

3. Basculez à nouveau sur la branche "develop" :

```
git checkout develop
```

4. Tentez de fusionner la branche "main" dans "develop" :

```
git merge main
```

5. Git signalera un conflit. Ouvrez le fichier README.md et résolvez le conflit en choisissant quelle modification conserver.

6. Après avoir résolu le conflit, ajoutez le fichier modifié :

```
git add README.md
```

7. Finalisez la fusion en effectuant un commit :

```
git commit -m "Résolution du conflit entre main et develop"
```

UTILISATION D'UN DÉPÔT DISTANT

Objectif : Connecter un dépôt Git local à un dépôt distant et effectuer des opérations de push et pull.

1. Créez un nouveau dépôt sur GitHub (ou une autre plateforme Git distante) sans initialiser de README, .gitignore ou licence.
2. Dans votre terminal, ajoutez le dépôt distant à votre dépôt local :

```
git remote add origin
```

```
git push -u origin main
```

3. Poussez vos commits locaux vers le dépôt distant :

```
git push origin main
```

4. Pour simuler une modification distante, allez sur GitHub et modifiez le fichier README.md directement dans l'interface web, puis validez les modifications.

5. Revenez à votre terminal et récupérez les modifications distantes :

```
git pull origin main
```

REVERT ET RESET

Objectif : Apprendre à annuler des modifications avec les commandes revert et reset.

1. Effectuez une modification dans le fichier README.md et validez-la :

```
git add README.md
git commit -m "Modification temporaire du README"
```

2. Utilisez la commande revert pour annuler ce commit tout en conservant l'historique :

```
git revert HEAD
```

3. Maintenant, effectuez une autre modification dans le fichier README.md et validez-la :

```
git add README.md
git commit -m "Autre modification temporaire du README"
```

4. Utilisez la commande reset pour annuler ce commit et revenir à l'état précédent (attention, cela supprimera le commit de l'historique) :

```
git reset --hard HEAD~1
```

VISUALISER L'HISTORIQUE DES COMMITS

Objectif : Utiliser les commandes Git pour visualiser l'historique des commits.

1. Utilisez la commande suivante pour afficher l'historique des commits avec des détails :

```
git log --oneline --graph --decorate --all
```

2. Explorez les options de la commande git log pour personnaliser l'affichage selon vos besoins.

```
git log --since="2 weeks ago" --author="VotreNom"
```

3. Essayez d'afficher les différences entre deux commits spécifiques en utilisant la commande git diff.

```
git diff <commit1> <commit2>
```

4. Utilisez la commande `git show` pour afficher les détails d'un commit particulier.

```
git show <commit-hash>
```

5. Expérimentez avec d'autres options de `git log`, comme `–since`, `–author`, ou `–stat`, pour filtrer et afficher l'historique des commits de différentes manières.

STASHING DES MODIFICATIONS

Objectif : Apprendre à utiliser la fonctionnalité de stashing pour sauvegarder temporairement des modifications non validées.

1. Effectuez des modifications dans le fichier README.md sans les valider.
2. Utilisez la commande suivante pour sauvegarder temporairement vos modifications :

```
git stash
```

3. Pour récupérer vos modifications mises de côté, utilisez la commande suivante :

```
git stash pop
```

CRÉATION ET UTILISATION DE TAGS

Objectif : Apprendre à créer et utiliser des tags dans Git pour marquer des versions spécifiques du code.

1. Créez un tag annoté pour marquer une version spécifique de votre projet :

```
git tag -a v1.0 -m "Version 1.0"
```

2. Listez tous les tags existants dans votre dépôt :

```
git tag
```

3. Poussez le tag vers le dépôt distant :

```
git push origin v1.0
```

4. Pour revenir à une version spécifique marquée par un tag, utilisez la commande `checkout` :

```
git checkout v1.0
```


CONFIGURATION DE GIT

Objectif : Configurer Git avec vos informations utilisateur et préférences.

1. Configurez votre nom d'utilisateur global :

```
git config --global user.name "VotreNom"
```

2. Configurez votre adresse e-mail globale :

```
git config --global user.email "VotreEmail@example.com"
```

IGNORER DES FICHIERS AVEC .GITIGNORE

Objectif : Apprendre à utiliser un fichier .gitignore pour exclure certains fichiers ou répertoires du suivi Git.

1. Créez un fichier nommé .gitignore à la racine de votre projet.
2. Ajoutez des règles pour ignorer certains fichiers ou répertoires. Par exemple, pour ignorer tous les fichiers .log et le répertoire temp/ :

```
*.log  
temp/
```

3. Enregistrez le fichier .gitignore et ajoutez-le au suivi Git :

```
git add .gitignore  
git commit -m "Ajout du fichier .gitignore"
```

COLLABORATION AVEC DES PULL REQUESTS

Objectif : Apprendre à collaborer avec des Pull Requests (PR) sur une plateforme Git distante comme GitHub.

1. Forkez un dépôt existant sur GitHub pour créer une copie dans votre compte.
2. Clonez votre fork localement :

```
git clone <projet-fork-url>
```

3. Créez une nouvelle branche pour vos modifications :

```
git checkout -b ma-nouvelle-fonctionnalite
```

4. Apportez des modifications au code, ajoutez et validez-les :

```
git add .  
git commit -m "Ajout d'une nouvelle fonctionnalité"
```

5. Poussez votre branche vers votre fork sur GitHub :

```
git push origin ma-nouvelle-fonctionnalite
```

6. Allez sur GitHub et créez une Pull Request depuis votre branche vers la branche principale du dépôt original.

REBASE VS MERGE

Objectif : Comprendre la différence entre rebase et merge dans Git.

1. Créez une nouvelle branche "feature" à partir de "main" et apportez des modifications.
2. Basculez sur la branche "main" et apportez des modifications différentes.
3. Revenez à la branche "feature" et essayez de fusionner "main" en utilisant merge :

```
git merge main
```

4. Résolvez les conflits s'il y en a, puis validez la fusion.
5. Maintenant, revenez à la branche "feature" et essayez de réappliquer les modifications de "main" en utilisant rebase :

```
git rebase main
```

6. Résolvez les conflits s'il y en a, puis continuez le rebase :

```
git rebase --continue
```

NETTOYAGE DU DÉPÔT AVEC GIT GC

Objectif : Apprendre à utiliser la commande git gc pour nettoyer et optimiser le dépôt Git.

1. Exécutez la commande suivante pour lancer le processus de nettoyage et d'optimisation :

```
git gc
```

2. Observez les messages affichés pour comprendre ce qui a été nettoyé ou optimisé.

UTILISATION AVANCÉE DE GIT LOG

Objectif : Explorer les options avancées de la commande git log pour une meilleure visualisation de l'historique des commits.

1. Utilisez la commande suivante pour afficher l'historique des commits avec des détails :

```
git log --oneline --graph --decorate --all
```

2. Explorez les options de la commande git log pour personnaliser l'affichage selon vos besoins.

```
git log --since="2 weeks ago" --author="VotreNom"
```

3. Essayez d'afficher les différences entre deux commits spécifiques en utilisant la commande git diff.

```
git diff <commit1> <commit2>
```

4. Utilisez la commande git show pour afficher les détails d'un commit particulier.

```
git show <commit-hash>
```

5. Expérimentez avec d'autres options de git log, comme --since, --author, ou --stat, pour filtrer et afficher l'historique des commits de différentes manières.

CONFIGURATION AVANCÉE DE GIT

Objectif : Explorer les options de configuration avancée de Git.

1. Configurez l'éditeur de texte par défaut pour Git (par exemple, Vim ou Nano) :

```
git config --global core.editor "vim"
```

2. Activez la coloration syntaxique dans le terminal pour une meilleure lisibilité :

```
git config --global color.ui auto
```

3. Configurez un alias pour une commande Git fréquemment utilisée (par exemple, git status) :

```
git config --global alias.st status
```

WORKSHOP

La programmation orientée objet (POO) consiste à définir des objets logiciels et à les faire interagir entre eux.

A1. NOTION DE CLASSE

Une classe est un "moule" ou un plan de construction. Elle définit les caractéristiques (attributs) et les actions (méthodes) communes à un ensemble d'objets.

```
public class Point
{
    public double X; // Attribut (ou champ)
    public double Y;
}
```

A2. NOTION D'OBJETS / INSTANCES

Un objet est une incarnation concrète d'une classe. En C#, on utilise le mot-clé `new` pour créer un objet en mémoire (tas/heap).

```
Point p1 = new Point(); // Instance de la classe Point
p1.X = 10.5;
```

A3. NOTION DE VISIBILITÉ

C# utilise des modificateurs d'accès pour contrôler la sécurité des données :

- **public** : accessible de partout.
- **private** : accessible uniquement à l'intérieur de la classe (par défaut).
- **protected** : accessible dans la classe et ses enfants (héritage).
- **internal** : accessible dans tout le projet (assembly).

A4. NOTION D'ENCAPSULATION (PROPRIÉTÉS)

En C#, on n'utilise presque jamais de champs publics. On utilise des Propriétés pour protéger les données.

La raison principale est le principe du "**Moindre Privilège**" :

- **Encapsulation** : On protège l'objet contre une utilisation incorrecte. Si tout était public par défaut, un développeur pourrait modifier une donnée interne critique de l'extérieur, brisant ainsi la logique de la classe.
- **Sécurité et Maintenance** : En cachant tout par défaut, vous forcez le concepteur de la classe à réfléchir consciemment à ce qu'il veut exposer (l'interface publique). Cela permet de modifier le code interne plus tard sans casser les programmes qui utilisent votre classe.

```
public class CompteBancaire
{
    private decimal _solde; // Champ privé (Backing field)

    public decimal Solde // Propriété
    {
        get { return _solde; }
        private set {
            if (value >= 0) _solde = value;
        }
    }
}
```

A5. LES RECORDS

Un **record** est une structure légère destinée à stocker des données. Contrairement aux classes, deux records sont considérés comme égaux si leurs valeurs sont identiques.

```
public record Personne(string Nom, int Age);
// Génère automatiquement constructeur, propriétés et comparaison
```

A6. TYPES VALEUR (STRUCT) VS RÉFÉRENCE (CLASS)

En C#, les types sont soit des **types valeur** (struct) soit des **types référence** (class).

- **Class** : Allouée sur le **Heap** (Tas). On manipule une adresse mémoire.
- **Struct** : Allouée sur le **Stack** (Pile). Plus rapide pour les petits objets immuables.

B1. CONSTRUCTEURS

Le constructeur initialise l'objet. Il porte le nom de la classe et n'a pas de type de retour.

```
public class Robot
{
    public string Nom { get; set; }

    // Constructeur
    public Robot(string nom)
    {
        this.Nom = nom; // 'this' désigne l'objet actuel
    }
}
```

B2. PARAMÈTRES OPTIONNELS

Au lieu de créer 3 constructeurs, C# permet de définir des valeurs par défaut.

```
public Point(double x = 0, double y = 0)
{
    X = x;
    Y = y;
}
// Appels possibles : new Point(), new Point(5), new Point(5, 10)
```

B3. RÉFÉRENCES ET GARBAGE COLLECTOR

En C#, contrairement au C++, vous ne gérez pas la destruction des objets (pas de delete). Le Garbage Collector (ramasse-miettes) libère automatiquement la mémoire quand l'objet n'est plus utilisé.

B4. LA DIRECTIVE USING

Le Garbage Collector ne ferme pas les fichiers ou les bases de données instantanément. Le bloc **using** garantit la libération immédiate des ressources externes.

```
using (var reader = new StreamReader("file.txt")) {
    // Le fichier est ouvert
} // Le fichier est fermé ici automatiquement
```

C1. MEMBRES STATIQUES

Le mot-clé **static** signifie que le membre appartient à la classe elle-même et non à une instance particulière.

```
public class Calculatrice
{
    public static double PI = 3.14159;

    public static double Somme(double a, double b) => a + b;
}

// Utilisation sans créer d'objet
double r = Calculatrice.PI;
```

C2. LINQ (LANGUAGE INTEGRATED QUERY)

LINQ permet de manipuler les listes de données comme si on faisait des requêtes SQL, de façon très concise.

```
var nomsLongs = listeNoms.Where(n => n.Length > 5)
                        .OrderBy(n => n)
                        .ToList();
```

D1. HÉRITAGE

En C#, on utilise l'opérateur : pour l'héritage. Une classe ne peut hériter que d'une seule classe parente.

```
public class Animal
{
    public virtual void Crier() => Console.WriteLine("L'animal crie");
}

public class Chien : Animal
{
    public override void Crier() => Console.WriteLine("Le chien aboie");
}
```

D2. POLYMORPHISME (VIRTUAL / OVERRIDE)

Pour qu'une méthode puisse être redéfinie, elle doit être marquée **virtual** dans le parent. L'enfant utilise **override**.

```
Animal monAnimal = new Chien();
monAnimal.Crier(); // Affiche "Le chien aboie" (Liaison tardive)
```

D3. CLASSES ABSTRAITES

Une classe abstract ne peut pas être instanciée. Elle sert de base à d'autres classes.

```
public abstract class Forme
{
    public abstract double CalculerAire(); // Pas de corps de fonction
}
```


D4. LES INTERFACES

Une interface est un contrat. Elle définit "ce que l'objet sait faire" sans dire "comment il le fait". Une classe peut implémenter plusieurs interfaces.

```
public interface IVolant
{
    void Voler();
}

public class Avion : IVolant
{
    public void Voler() { /* Code pour décoller */ }
}
```

E1. ASYNC / AWAIT

Pour éviter de figer l'interface utilisateur ou bloquer un serveur, on utilise des tâches asynchrones. Le mot-clé **await** suspend l'exécution sans bloquer le thread en attendant la fin d'une opération (ex: réseau).

```
public async Task<string> GetDataAsync() {
    var result = await _httpClient.GetStringAsync("https://api.com");
    return result;
}
```

ANALYSE ET RÉOLUTION : BURNOUT

L'entreprise 2F Roaming a connu une croissance rapide, passant de 2 à 20 développeurs, sans adapter ses méthodes de travail. Ce manque de structuration a entraîné des problèmes de qualité logicielle, de productivité et de gestion des versions. Pour répondre à cette problématique, il est nécessaire d'adopter une démarche structurée et progressive, centrée sur l'amélioration continue et l'intégration de pratiques modernes du génie logiciel.

Résolution proposée :

- **Mise en place d'un système de gestion de versions moderne (Git)** : Permettre un suivi précis des modifications, faciliter la collaboration et garantir la traçabilité du code.
- **Adoption de la Programmation Orientée Objet (POO)** : Structurer le code pour le rendre plus modulaire, maintenable et évolutif.
- **Automatisation des tests** : Intégrer des tests unitaires et d'intégration pour assurer la qualité du logiciel à chaque évolution.
- **Standardisation de l'environnement de développement** : Définir des outils et des configurations partagés (IDE, extensions, règles de codage) pour homogénéiser les pratiques.
- **Formation et accompagnement de l'équipe** : Organiser des sessions de formation sur les nouveaux outils et méthodes, et instaurer une culture DevOps favorisant la communication et l'amélioration continue.
- **Mise en place de l'intégration continue (CI)** : Automatiser les processus de build, de test et de déploiement pour accélérer les livraisons et réduire les erreurs humaines.

Cette démarche permettra à 2F Roaming d'améliorer la qualité de ses livrables, d'accroître la productivité de ses équipes et de mieux gérer la croissance de son effectif.

CONCLUSION

La modernisation des pratiques de développement chez 2F Roaming est indispensable pour accompagner la croissance de l'entreprise. En adoptant des outils adaptés (gestion de versions, automatisation des tests, CI), en structurant le code (POO) et en formant les équipes, l'entreprise pourra garantir la qualité, la productivité et la pérennité de ses projets logiciels. L'implication de tous les membres et le soutien de la direction seront les clés du succès de cette transformation.

CAPITALISATION

AAV	Commentaire	Statut
Décrire .NET Core et .NET Framework et leur rôle dans .Net	Les différences, rôles et évolutions sont expliqués, avec schéma et exemples.	Acquis
Décrire les architectures .NET	L'architecture interne (CLR, CoreCLR, composants) est détaillée, distinctions claires.	Acquis
Expliquer les évolutions entre .NET Core et .NET Framework	Les ruptures (performance, modularité, open source) sont bien identifiées et justifiées.	Acquis
Expliquer les notions de CoreCLR et CLR	Explications précises sur le CLR/CoreCLR, compilation, JIT, Garbage Collector.	Acquis
Expliquer les notions d'assemblage pour .NET Core et .NET Framework	Assemblages, GAC, déploiement autonome bien expliqués, différences mises en avant.	Acquis
Connaître les principaux outils supportant la chaîne d'intégration continue	Outils CI (Azure DevOps, GitHub Actions, GitLab CI, SonarQube) listés et expliqués.	Acquis
Connaître les principes généraux de la conteneurisation et ses avantages	Principe de conteneurisation, avantages et outils (Docker, Dockerfile) présentés clairement.	Acquis
Connaître les environnements et outils Docker	Docker Desktop, Docker Hub, fichiers Dockerfile mentionnés et contextualisés.	Acquis
Pratiquer l'installation et la configuration de Git	Procédure d'installation, configuration utilisateur, commandes de base détaillées.	Acquis
Relier Visual Studio à un dépôt Git	Lien entre Visual Studio et Git expliqué, utilisation de l'onglet "Git Changes" abordée.	Acquis
Illustrer les principes généraux de gestion de versions pour un fichier et Appliquer les commandes Git associées	Principes de versionning, commandes Git (init, add, commit, push, pull, merge, etc.) illustrés par des exemples pratiques.	Acquis

RESSOURCES

- **Documentation officielle de Git** : <https://git-scm.com/doc>
- **Pro Git Book** : <https://git-scm.com/book/en/v2>
- **GitHub Learning Lab** : <https://lab.github.com/>
- **POO** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/fundamentals/object-oriented/>
- **C# Guide** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/>
- **LINQ** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/programming-guide/concepts/linq/>
- **Async/Await** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/programming-guide/concepts/async/>